

•



- [Core Concepts](#)
- [Plans](#)
- Data Partitioning for the Metrics Operator

On this page

Data Partitioning for the Metrics Operator

Was this page helpful? 

Data partitioning is a key step for processing distributed jobs in a Spark cluster spawned by [plan executions](#) to enable Pulse to handle large volumes of data efficiently.

When you add [metrics operators](#) to a [plan](#), Spark needs to re-organize the data before calculating the metrics defined in each metrics operator. During this data re-organization, all the rows (e.g., in fraud applications each row corresponds to a transaction) that share the same value in the fields that are configured as *Group By* in the metrics operator are placed within the same Spark partition.

Pulse allows you to configure data partitioning using flexible UI options to ensure that Spark performs data re-organization as homogenous and well distributed as possible.

How does it work?

To get the best out of the data partitioning options available in the UI, it is important that you understand the logic behind this concept.

When you complete the configurations of your plan with all the required operators, you can generate the output data source by running a plan execution. The plan execution spawns distributed Spark jobs that are responsible for handling the heavy work so that Pulse can process large volumes of data (i.e. often hundreds or even thousands of TBs).

Spark streams data through its graph of operators, which maps reasonably well to the graph of operators that you configure in the plan's workspace that is available in the Pulse UI. This means that, in general, every row of the input data (e.g. in fraud applications each row corresponds to a transaction) is fully processed from the source to the sink.

If you have a plan with metrics operators, **the following data partitioning logic is applied** when you run the plan execution:

1. Spark streams all the data and process upstream operators (i.e. processes all the data that flows through the operators that precede the metrics operator).
2. The data that arrives at the metric operator is re-organized according to each set of *Group By* fields (e.g. Set 1: *card_number* + *merchant*, Set 2: *city*) defined in this operator. This process also is known as shuffling in the Spark terminology. For each set of *Group By* fields, all the rows that share the same value in the fields as defined in the *Group By* are placed within the same Spark partition.

For example, consider that you define a set of *Group By* with the fields *card_number* + *merchant*.

All the transactions that share the same Card Number AND the same Merchant are placed within a single Spark partition. The example shown below uses only the *Merchant* as the *Group By* field to provide a simple visualization of this process. Note that one Spark partition can (and will likely) contain data from various sets of *Group By* fields.

3. Pulse takes each partition and feeds it in streaming to PKernel, which is the Feedzai's stream processor for profile calculation. In this step, PKernel is responsible for calculating the metric defined in the metrics operator using data organized by *Group By* as explained in the previous step.
4. Steps 2 and 3 are repeated for each set of *Group By* fields so that all metrics are covered with a shuffle. For instance, if you have two metrics, the step sequence 2-3 is repeated twice as shown in the example below:
 - Iteration #1 (e.g. Metric 1: *card_number* + *merchant*): Spark re-organizes all the data by *Card Number* AND *Merchants*, and then PKernel performs the profile calculations.
 - Iteration #2 (e.g. Metric 2: *city*): Spark re-organizes all the data by *City*, and then PKernel performs the profile calculations.

The challenge of unbalanced data

A typical data partitioning method is to hash the values of each row and then send the rows that share the same hashed values to the same partition. However, when dealing with unbalanced data this method might introduce some inefficiency.

For example, imagine that you configure a metrics operator with a metric that has the *Group By* field *Merchant*. Now assume that you have two large merchants (Retailer 1, Retailer 2) each of which represents 1/3 of the total data, and then the remaining 1/3 of the data is a long-tail of very small merchants (Merchant 1, Merchant 2, ... , Merchant N). If you use the hash approach to partitioning data, there is nothing from preventing the most popular values from falling in the same partition:

This unbalanced data distribution is oblivious to the population and would require extra memory in the Spark workers to compute the data that is shown in Partition 1.

Continuing with the example above, an efficient way of dividing the data into different partitions would be the following one:

The data-aware optimizer available in Pulse allows you to set a plan execution that spawns an efficient data partitioning to maximize the memory allocated to the Spark workers. This option is specially advantageous when you are dealing with unbalanced data because it takes into account the popularity of each set of values for *Group By* fields.

Data partitioning options

Pulse has the following data partitioning options available:

- hash-based partitioning
- manual partitioning
- data-aware optimizer

The data-aware optimizer is the default option for data partitioning, but you can configure a different partitioning method to be used by default in the main configuration file `$PULSE_HOME/conf/pulse.properties`, under the property `pulse.global.ui.plans.datapartitioning.method`.

The available values for this key are `data-aware`, `hash-based` and `manual`.

Hash-based

The **hash-based** option does not take into account the popularity of each set of *Group By* fields when performing the data re-distribution into different partitions.

Additionally, this method does not optimize the number of partitions used by the shuffle. In fact, the hash-based method sets the same number of partitions used in the plan input. This number is obtained by reading chunks of 128MB from the input and making each chunk a partition.

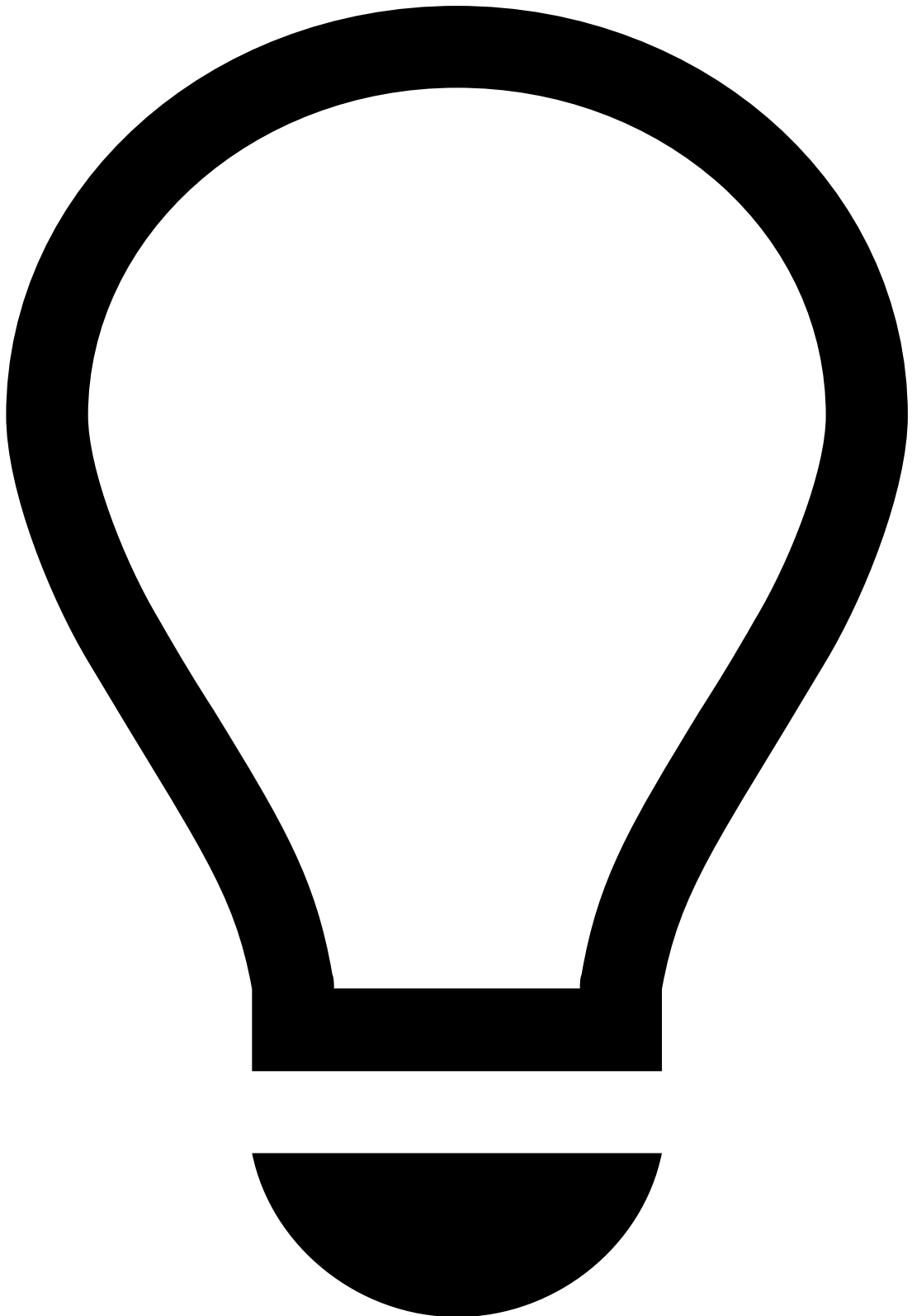
On the plus side, this method is less computationally expensive than the popularity aware method explained below. This option might be beneficial when you are computing metrics that have *Group By* fields that are expected to lead to a balanced data distribution.

Check the pages [Creating a stratified sample](#) and [Creating metrics](#) to learn how to choose the hash-based option.

Manual Partitioning

The **manual partitioning** option allows you to specify the number of partitions to which the values of your dataset are distributed. Manual partitioning uses the hash-based approach to distribute the rows with the same values to the same partition according to each set of *Group By* fields.

Check the pages [Creating a stratified sample](#) and [Creating metrics](#) to learn how to choose the manual partitioning option.



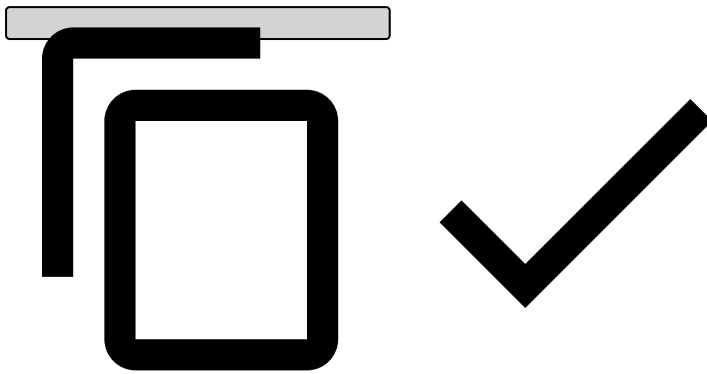
Configuration of manual partitioning

When selecting the `manual` partitioning option, you can manually set the number of partitions to a custom value in the Pulse UI. The default number of partitions is 100.

If you regularly need to specify a different number of partitions, you can set a custom number of partitions via configuration.

For example, to use manual partitioning with 20 000 partitions, you can set the following properties in the main configuration file `$PULSE_HOME/conf/pulse.properties` :

```
pulse.global.ui.plans.datapartitioning.method=manual  
pulse.global.ui.plans.datapartitioning.partitions=20000
```



Data-aware optimizer

The **data-aware optimizer** option takes into account the popularity of the values according to each set of *Group By* field.

The number of partitions used by the shuffle are dynamically computed, and they adapt to the actual data. The data-aware optimizer uses the minimum number of partitions that allows for a homogeneous distribution of the data that respects the set of *Group By* values.

The data-aware optimizer provides the following configurable options:

- **Most important K cut-off:** The number of the most popular entities (i.e. a set of equal values) that are tracked for collecting statistics and re-organizing data into different partitions. You can choose a large *K cut-off* value (e.g. 50 000).
 - Consider the example in which you configure a metrics operator with a metric that has the *Group By* field *Merchant*. Assume again that you have two large merchants (Retailer 1, Retailer 2) each of which represents 1/3 of the total data, and the remaining 1/3 of the data is a long-tail of very small merchants (Merchant 1, Merchant 2, ... , Merchant N). If you choose the k cut-off value equal to 2: the data associated with Retailer 1 and 2 will be re-organized using then data-aware optimizer option, whereas the remaining rows are re-organized with the hash-based approach.
- **Maximum rows per partition:** The number of maximum rows that fit on each partition. For example, if you know that each row in your dataset averages to 1KB, and that your Spark executors have 4 GB of heap for data (because they have 12 GB available for the executor, e.g.), then it is acceptable to configure 4 million rows per partition.

Check the pages [Creating a stratified sample](#) and [Creating metrics](#) to learn how to choose the data-aware optimizer option.

Computing statistics

Pulse logs statistics into the logging files of the job when you run a plan execution with the data-aware optimizer option. This helps data scientists to know how the data is being partitioned and, in case of failing to compute a plan, it might also help them to understand the reason behind the failure.

For each set of *Group By* fields that is shuffled (e.g. set1: "card-number"+"merchant"; set2: "device-id"), the following data is computed:

- Median
- Average
- Top 10 most predominant set of *Group By* field values and the corresponding cardinality.

This statistics can be accessed in the log folder, using the logging level DEBUG.